

LOOP — Agent Build Brief

Agents for Humans Hackathon · Everyday Agents track · Strands Agents SDK on Amazon Bedrock

Deadline: September 14, 2026, 5:00 PM PDT (= September 15, 2026, 12:00 AM UTC)

Instructions for the Agent Reading This Document

You are being handed a fully specified product and engineering brief. Every product decision in this document (scope, architecture, tool list, data model, UI, demo plan) is already made and is not up for debate or re-scoping. Do not ask the user to re-confirm the product direction, brainstorm alternatives, or propose a different architecture. Your job is to build exactly what is specified below, inside a very tight timeline (see Deadline above).

When the user tells you to begin, do the following, in order:

- Ask a single, short batch of setup/configuration questions — the list below, nothing more. Do not ask about product scope, features, naming, or design; those are already decided.
- Wait for the answers.
- Then start building immediately, without further discussion, clarification rounds, or checking in before each step. Work through the Build Priority Order in Section 12 top to bottom.
- Only interrupt the user again if you hit a genuine blocker (a missing credential, a library that will not install, an ambiguity this document does not resolve) — not for approval of routine implementation choices.

The setup questions to ask (ask all of these together, in one message, not one at a time):

- Language: Python or TypeScript for the Strands Agents SDK?
- AWS access: is an AWS account with Bedrock model access already available, or does the agent need to walk the user through account setup and requesting model access first?
- AWS region: which region should Bedrock and (optionally) DynamoDB run in?
- Storage: DynamoDB, or a local JSON file for the Life Events store? (JSON is the safe fallback given the timeline — see Section 8.)
- UI approach: a minimal Streamlit app, a plain Flask+HTML page, or something else the user already has tooling for? Optimize for speed to a working demo, not visual polish.
- Repository: name, and confirm it will be public with an MIT or Apache-2.0 license (both required by the hackathon rules — see Section 11).
- AgentCore: attempt AgentCore Runtime deployment for the scoring bonus, or explicitly skip it given the timeline? Default answer if the user is unsure: skip it, revisit only if everything else works with time to spare.

1. Project Overview

One-line pitch: LOOP is an autonomous life-admin agent that discovers unfinished business hiding in a person's everyday information — receipts, bills, complaint emails — and turns it into completed actions, not another reminder.

LOOP is being built for the **Everyday Agents** track of the Agents for Humans Hackathon (Devpost, AWS-sponsored). The track's own definition: an agent that takes the busywork out of daily life, home, money, health, errands, or family — the best ones run quietly in the background and only surface when there is a real decision for the person to make. LOOP is designed around that definition directly.

2. The Problem

A 2024 *Public Administration Review* study (Martin, 2,243 UK adults surveyed) found people spend roughly 32 to 85 minutes a day on administrative tasks across ten life domains — bills, health, benefits, debt, and more, depending on measurement method. This is a real, citable figure; use it verbatim in the submission's text description rather than inventing a different number.

Existing products in this space (organizers, reminder apps, subscription trackers) manage information. None of them close the loop: they stop at “here's a reminder” and leave the actual follow-through to the user. That gap — between being reminded and something actually getting resolved — is what LOOP exists to close.

3. What LOOP Does Differently

Reminder apps notify. LOOP investigates, prepares the work, and — with approval — acts. The core agent loop, and the language to use consistently across the pitch, README, and demo narration:

DETECT -> UNDERSTAND -> RESEARCH -> PREPARE -> ASK -> EXECUTE

- **Detect** — notice a new piece of information (a receipt, an email, a bill) that might matter
- **Understand** — work out what it actually means (this is a warranty; this bill is unusually high)
- **Research** — look up what can be done about it (policy terms, procedures)
- **Prepare** — do the legwork: draft the claim, assemble the evidence
- **Ask** — stop and request human approval before anything high-impact happens
- **Execute** — carry out the approved action and record it

4. MVP Scope — Two Open Loops, Built Completely

The original concept identifies five open-loop categories (money to lose, money to recover, deadlines, submissions, recurring commitments). Building all five in a ~4-day solo build means none of them work well. **The MVP demonstrates exactly two, fully built and polished.** The remaining three are real and belong in the pitch's roadmap section, not in the working build (see Section 13).

4.1 Flagship case — Warranty claim (“money you're entitled to recover”)

This is the hero moment of the whole demo. Build this one end to end and make it flawless before touching anything else. Full walkthrough:

- 1. **Detect** — a new receipt document arrives for a pair of headphones
- 2. **Understand** — extract product, price, purchase date, and warranty length; create a structured Life Event (schema in Section 6)
- 3. **Detect** (later, simulated as "days later" in the demo data) — a new email arrives: "my headphones keep shutting down"
- 4. **Understand** — recognize this email may relate to the earlier Life Event
- 5. **Research** — look up the warranty/return policy (reference data, not a live web lookup)
- 6. **Prepare** — draft the warranty claim, attach the receipt
- 7. **Ask** — "Warranty claim ready — submit?"
- 8. **Execute** (on approval) — mark the claim as submitted, log it in the activity timeline

4.2 Second case — Bill anomaly (“money you're about to lose”)

Lighter weight than the warranty case; its job is to prove the Life Events model generalizes beyond one scenario, without requiring new tooling.

- Compare a new bill's amount to the user's historical average for that provider
- Flag it as an anomaly if the increase is significant (e.g. >15–20%)
- Prepare a short, plain-language explanation of the likely cause if one is discoverable in the reference data (e.g. a rate change), otherwise note that the cause is unclear and flag it for the user to investigate
- This case does not need to reach "execute" — surfacing the anomaly with a clear explanation is enough to demonstrate the model

5. Explicitly Out of Scope for This Build

- Real email/calendar/bank integrations (OAuth, live accounts) — replaced entirely by a synthetic demo dataset shipped in the repo (see Section 7)
- Doctor-appointment form-filling and insurance-renewal flows — same underlying mechanic as the warranty case, cut for time
- Five specialized sub-agents (a separate detection/research/document/action agent each) — replaced by **one** orchestrator agent using six tools. Same reasoning loop, far less to build and debug
- Actually submitting a claim or cancellation to a real third party — the execute step marks the action as done inside LOOP's own state; it does not call an external company's real system
- User accounts, multi-tenant auth, or a login system — a single hardcoded demo user is correct and sufficient for a hackathon submission

6. Data Model — Life Events, Not Documents

This is the central architectural idea: documents are evidence, Life Events are what the agent actually reasons about. Do not store and reason over raw documents directly — always convert them into a structured Life Event first. Two event types are needed for the MVP:

```
{
  "type": "warranty",
  "product": "Sony Headphones",
  "purchase_date": "2026-03-14",
  "expires": "2027-03-14",
  "receipt": "receipt_392.pdf",
  "status": "active",
  "linked_signal": null,
  "potential_action": "warranty_claim"
}

{
  "type": "bill",
  "provider": "Electricity",
  "current_amount": 187,
  "historical_average": 147,
  "anomaly": true,
  "potential_action": "investigate"
}
```

Every Life Event should also carry a **status** field that moves through a simple lifecycle as the agent works on it: detected -> investigating -> prepared -> awaiting_approval -> executing -> completed (or dismissed). The activity timeline in the UI is a direct rendering of these status transitions over time, so log a timestamped entry every time a Life Event's status changes.

7. Synthetic Demo Dataset

Build a small, realistic, fictional dataset and ship it as plain text/JSON files in the repository. Do not build any real integration to source this data. Minimum required content:

- One receipt document for an electronics purchase (e.g. headphones or a laptop) with product, price, purchase date, seller, and warranty length clearly stated
- One complaint email, dated after the receipt, describing a plausible product fault, written the way a real person would write it (not a structured form)
- One warranty/return policy reference document the `lookup_policy` tool can consult (can be a short static text file, not a live source)
- A short history of electricity bills (4–6 months) with one recent bill noticeably higher than the historical average
- Optionally, 2–3 additional unrelated documents/emails (a subscription renewal notice, a delivery confirmation) purely as background clutter, so the dashboard's "we scanned your messy digital life" opening beat has something to show — these do not need their own working Life Event logic

Keep all of this as flat files (text or JSON) inside the repo, e.g. under `/demo_data/`. This is what `get_documents` reads from.

8. Architecture

One Strands orchestrator agent — not five separate agents. It reasons over the loop in Section 3 using six tools:

- **get_documents** — returns the synthetic receipts/emails/bills for the demo user
- **lookup_policy** — returns reference warranty/return-window rules
- **compare_bill** — compares a bill to the historical average and flags anomalies
- **draft_action** — prepares a claim or cancellation draft
- **request_approval** — Strands human-in-the-loop checkpoint before any high-impact action
- **execute_action** — marks the approved action done in the Life Events store

Storage: DynamoDB for the Life Events store, falling back to a local JSON file if DynamoDB setup eats into build time — state which one was actually used, plainly, in the README. **Documents:** plain text/JSON files shipped in the repo, per Section 7. No OCR/Textextract pipeline is needed — there is nothing to scan, only to read. **Model:** Amazon Bedrock, called through Strands. **Deployment (optional, last):** AgentCore Runtime, only attempted once the core loop works end to end and time remains — it is a scoring bonus, not a requirement.

LOOP — System Architecture

Agents for Humans Hackathon (Everyday Agents track), built with Strands Agents SDK on Amazon Bedrock

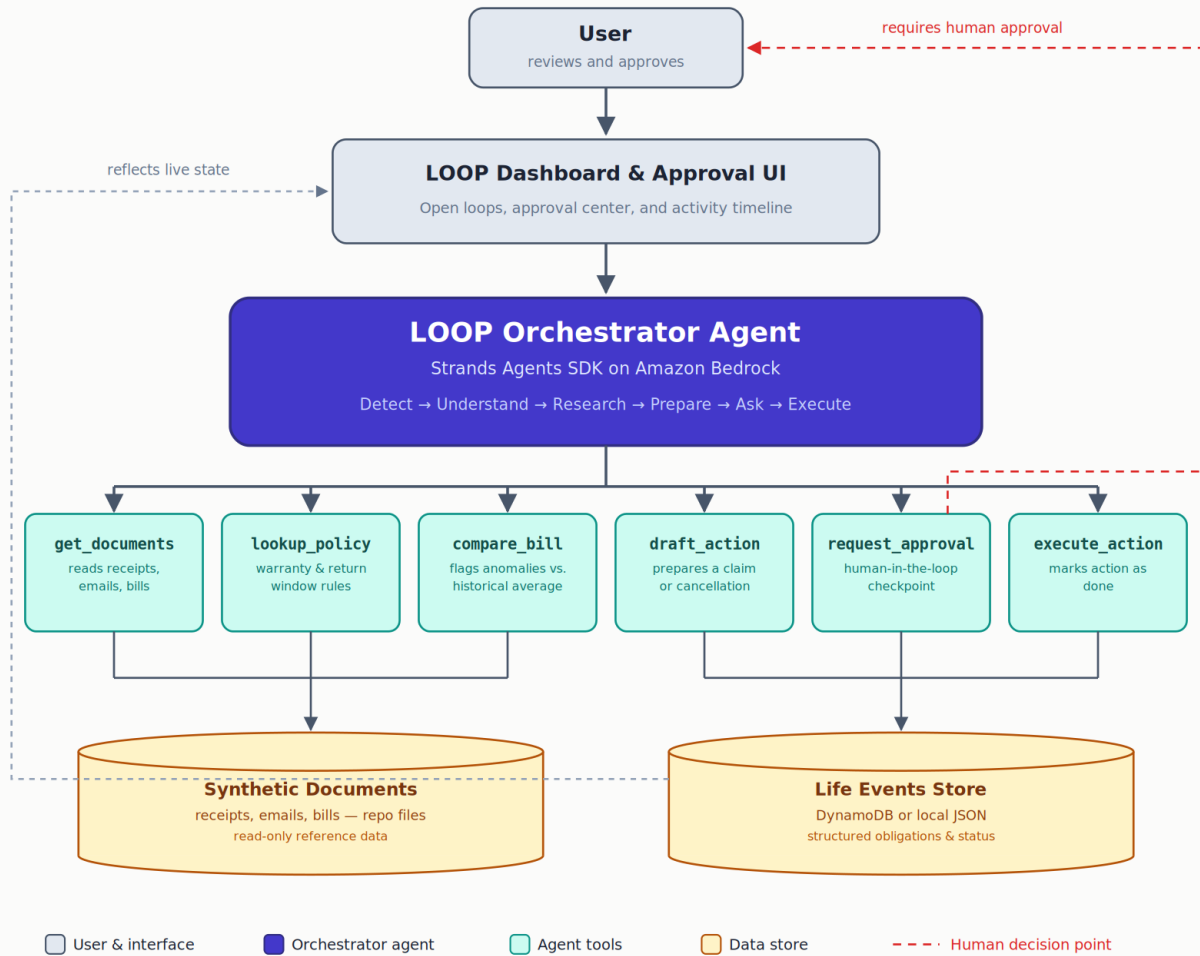


Figure 1. LOOP system architecture.

9. Human Approval Rules

Not everything the agent does needs a human in the loop. Split actions into two tiers:

Low-risk (can happen automatically, no approval needed):

- Reading a document, extracting information, creating or updating a Life Event, researching a policy, comparing a bill, preparing a draft

High-impact (must pause for approval via `request_approval`):

- Submitting a claim, cancelling a subscription, sending a message on the user's behalf, or any other action that is hard to undo

This is the philosophy to keep repeating in the pitch: autonomous by default, human when it matters.

10. UI Specification

Three screens. No auth system — a single hardcoded demo user is correct for this submission.

- **Dashboard** — "LOOP found N open loops." One card per open Life Event, showing type, a one-line plain-language summary, and a dollar amount where relevant (e.g. potential recovery amount)
- **Approval view** — opened from a dashboard card. Shows the evidence the agent gathered (which documents, what it concluded), the prepared action, and three buttons: Approve / Edit / Reject
- **Activity timeline** — a chronological log of every status change the agent made, timestamped, in plain language (e.g. "8:44 PM — Detected potential product issue")

Build whichever of these is fastest for the chosen stack to make look clean on camera — a Streamlit app is a reasonable default choice given the timeline.

11. Submission Checklist

- Text description: what LOOP does, who it's for, and how it works
- Public code repository — MIT or Apache-2.0 license visible in the About section, plus a README with setup instructions
- Architecture diagram (Figure 1 above, or export a fresh copy from the accompanying SVG/Mermaid source)
- Demo video, maximum 5 minutes. Pitch must cover: (1) the problem being solved, (2) who it's for, (3) why it matters. Slides, screen recordings, and voiceover are fine — no need to appear on camera
- AWS Builder ID
- Optional live demo link — scores higher on Technical Implementation if included, but is not required

12. Demo Script (target: under 5 minutes)

Time	Beat
0:00 - 0:10	Show the messy synthetic inbox/documents

0:10 - 0:30	Run LOOP -> dashboard shows the open loops found, including the recoverable warranty and the bill anomaly
0:30 - 1:30	Open the warranty card -> show the reasoning trail connecting receipt + complaint email + policy
1:30 - 2:00	Approve the claim -> show it execute and move to Completed
2:00 - 2:30	Quick look at the bill anomaly card to show the model generalizes beyond one case
2:30 - 3:30	Close on problem / who it's for / why it matters, exactly as the submission rules require

13. Roadmap (for the pitch's ambition, not the build)

State clearly in the text description and demo narration that this is deliberate scoping, not a limitation the team failed to overcome. The three categories below are the natural next steps once the core reasoning loop is proven in this build:

- Deadlines you're about to miss (renewals, returns, appointments, document submissions)
- Things you need to submit or respond to (forms, applications, confirmations)
- Recurring commitments worth reconsidering (subscriptions, memberships, insurance)

14. Tech Stack Summary

- **Strands Agents SDK** (Python or TypeScript, Apache-2.0) — required by the hackathon. Model-driven: define a model, a system prompt, and tools; the SDK runs the reasoning loop. Has native human-in-the-loop support — use it directly for `request_approval` rather than building an approval mechanism from scratch
- **Amazon Bedrock** — the model backing the agent's reasoning
- **DynamoDB or local JSON** — the Life Events store; choose based on the setup answers above
- **Amazon Bedrock AgentCore** (optional, last) — a managed runtime for deploying the finished agent (session isolation, long-running sessions). Framework-agnostic; works with whatever was built in Strands. Only attempt this after the core loop works end to end
- Explicitly not needed: Textract, Cognito, SES, EventBridge, CloudWatch, or separate Lambda functions. None of these are visible in a 5-minute demo video and none are worth the build hours

15. Build Priority Order

Work through this list in order. Do not move to a later item until the current one is functionally working.

1. Repo skeleton: folder structure, chosen license file (MIT or Apache-2.0), README stub
2. Life Events data model and storage (JSON file first; upgrade to DynamoDB only if time allows)
3. Synthetic dataset (Section 7) written to `/demo_data/`
4. The six tools, implemented and unit-tested against the synthetic dataset directly (no agent yet)
5. The Strands orchestrator agent: system prompt implementing the Detect -> Understand -> Research -> Prepare -> Ask -> Execute loop, wired to the six tools, with `request_approval` as a real human-in-the-loop checkpoint
6. Run the warranty-claim flagship case end to end via the agent; fix until it is flawless
7. Run the bill-anomaly case end to end; fix until it works
8. Minimal UI: dashboard, approval view, activity timeline (fastest stack available, per the setup answers)
9. Record the demo video against the script in Section 12; check it is under 5 minutes
10. Write the text description, finalize the README, confirm the license is visible, push the public repo
11. Only if time remains: attempt AgentCore Runtime deployment for the scoring bonus

16. Definition of Done

The MVP is complete when all of the following are true:

- A user can run LOOP against the synthetic dataset and see it discover both the warranty opportunity and the bill anomaly without being told to look for them
- The warranty case runs the full Detect -> Understand -> Research -> Prepare -> Ask -> Execute loop, pausing for real human approval before Execute
- The activity timeline shows a truthful, timestamped record of what the agent actually did
- Every item in the Submission Checklist (Section 11) is satisfied
- The demo video runs under 5 minutes and covers the problem, who it's for, and why it matters